Section 15: Trees

This section is part of the 5-hour LeetCode DSA Foundations course by Mukul Raina

- YouTube Course: https://youtu.be/YJMBv4Q2Tg8?si=Q1vDzusUJf753GG3
- YouTube Channel: https://www.youtube.com/@mukulraina
- Instructor LinkedIn: https://www.linkedin.com/in/mukulraina/

C11: LeetCode DSA Foundations for Interviews

Section 15: Trees Foundations

- SLIDE 138: Section Intro, Trees
- SLIDE 139: What Is a Tree and Binary Tree Structure
- SLIDE 140: Binary Tree Traversals: Inorder, Preorder, Postorder
- SLIDE 141: Level-Order Traversal (BFS on Trees)
- SLIDE 142: Recursive Tree Patterns
- SLIDE 143: Worked Example: Maximum Depth of Binary Tree
- SLIDE 144: Binary Search Trees: Properties and Operations
- SLIDE 145: Worked Example: Validate Binary Search Tree
- SLIDE 146: Wrong vs Right: Tree Mistakes
- SLIDE 147: How to Present Tree Solutions in an Interview
- SLIDE 148: Interview Practice: Trees
- SLIDE 149: Section Cheat Sheet

C11: LeetCode DSA Foundations for Interviews

Section 15: Trees Foundations

SLIDE 138: Section Intro, Trees

Section 15: Trees Foundations

Trees appear in roughly 25-30% of DSA interviews. Despite how many problems exist in this category, nearly every tree problem reduces to a traversal variant combined with a recursive pattern. Once you can traverse a tree and apply the three recursive patterns this section teaches, you can solve the majority of tree problems at the foundations level.

The most important insight about trees: a tree is a recursive data structure. Every subtree is itself a tree. This is why recursion from Section 12 is the primary tool for tree problems. If you understood the "trust the recursion" principle there, you already have the mental model you need here.

What We'll Cover:

1. Binary tree structure and terminology
2. Four traversal types: inorder, preorder, postorder (depth-first), and level-order (breadth-first)
3. Three recursive patterns that solve most tree problems
4. Binary search tree properties, search, and insertion
5. Worked examples: maximum depth, validate BST
6. How to present tree solutions in interviews

Where This Fits in the Course:

Section 12 (Recursion and Backtracking) built the recursive thinking you need for trees. Section 10 (Stacks and Queues) introduced the queue-based BFS pattern, which this section applies to trees as level-order traversal. This section is prerequisites for Section 16 (Graphs), where BFS and DFS generalize from trees to arbitrary graph structures with cycles and multiple components.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

SLIDE 139: What Is a Tree and Binary Tree Structure

Tree Definition

A tree is a connected, acyclic graph. "Connected" means every node is reachable from every other node. "Acyclic" means there are no loops: there is exactly one path between any two nodes.

A binary tree is a tree where each node has at most two children, called left and right.

Node Structure in Java:

```
public class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;

    TreeNode(int val) {
        this.val = val;
    }
}
```

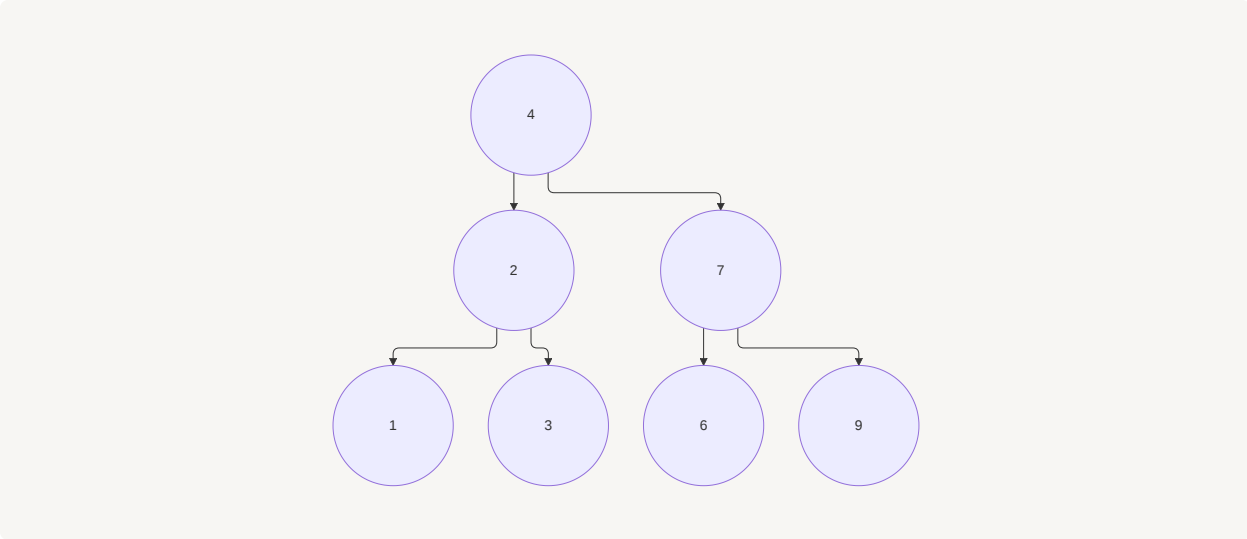
Each node holds a value and references to its left and right children. A null reference means no child exists at that position. A leaf node is one where both left and right are null.

Key Terminology:

Term	Definition	Example
Root	The topmost node, has no parent	Node 4 in the tree below
Leaf	A node with no children (left and right are both null)	Nodes 1, 3, 6, 9
Depth of a node	Number of edges from the root to that node	Root has depth 0, its children have depth 1

Term	Definition	Example
Height of a node	Number of edges from that node to its deepest leaf descendant	A leaf has height 0
Height of the tree	Height of the root node	Equals the maximum depth of any leaf
Subtree	A node and all its descendants	The left subtree of 4 is the tree rooted at 2
Complete binary tree	Every level is fully filled except possibly the last, which fills left to right	Heaps use this structure
Balanced binary tree	Height is O(log n), left and right subtree heights differ by at most 1	AVL trees, red-black trees

Example Tree:



This tree has 7 nodes, height 2, and 4 leaf nodes. Node 4 is the root. The left subtree of 4 contains {2, 1, 3}. The right subtree contains {7, 6, 9}.

The Recursive Property

Every subtree is itself a valid tree. The left subtree rooted at node 2 is a tree with root 2, left child 1, and right child 3. This is why recursion works naturally on trees: you can solve a problem for the full tree by solving it for the left and right subtrees and combining the results.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

SLIDE 140: Binary Tree Traversals: Inorder, Preorder, Postorder

Three Depth-First Traversals

The three traversals differ only in when the current node is processed relative to its children. "Process" means whatever operation the problem requires: printing, summing, collecting, comparing.

Traversal	Order	Mnemonic	Primary Use
Inorder	Left, Root, Right	L-N-R	Produces sorted order for BSTs
Preorder	Root, Left, Right	N-L-R	Copies or serializes tree structure
Postorder	Left, Right, Root	L-R-N	Bottom-up computation (height, size, deletion)

Full Java Code for All Three:

```
public void inorder(TreeNode node, List<Integer> result) {
    if (node == null) return;
    inorder(node.left, result);
    result.add(node.val);
    inorder(node.right, result);
}

public void preorder(TreeNode node, List<Integer> result) {
    if (node == null) return;
    result.add(node.val);
    preorder(node.left, result);
    preorder(node.right, result);
}

public void postorder(TreeNode node, List<Integer> result) {
    if (node == null) return;
    postorder(node.left, result);
```

```
        postorder(node.right, result);
        result.add(node.val);
    }
```

The structure is identical in all three. The only difference is where `result.add(node.val)` appears: before both recursive calls (preorder), between them (inorder), or after both (postorder).

Trace Through the Example Tree [4, 2, 7, 1, 3, 6, 9]:

```
Tree structure:
        4
       / \
      2   7
     / \ / \
    1  3 6  9

Inorder  (L, N, R):  1, 2, 3, 4, 6, 7, 9   ← sorted order
Preorder (N, L, R):  4, 2, 1, 3, 7, 6, 9   ← root first
Postorder(L, R, N):  1, 3, 2, 6, 9, 7, 4   ← root last
```

Why Inorder on a BST Gives Sorted Order:

In a BST, every value in the left subtree is smaller than the root, and every value in the right subtree is larger. Inorder visits left (all smaller values) first, then the root, then right (all larger values). This produces values in ascending order. If you run inorder on a BST and the result is not sorted, the tree is not a valid BST. This fact is used directly in Slide 145 (Validate BST).

Complexity for All Three Traversals:

Time O(n): every node is visited exactly once. Space O(h): the recursion stack depth equals the height of the tree. For a balanced tree h = O(log n). For a completely skewed tree (linked list shape) h = O(n).

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

SLIDE 141: Level-Order Traversal (BFS on Trees)

The Fourth Traversal: Breadth-First

Level-order traversal visits nodes level by level, left to right, top to bottom. Unlike the three depth-first traversals (which use recursion and the call stack), level-order uses a queue.

This is the tree version of the BFS skeleton introduced in Section 10 (Slide 94). The key difference: trees have no cycles, so no visited set is needed. In Section 16 (Graphs), the visited set comes back because graphs can have cycles.

Full Java Code:

```
public List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result;

    Deque<TreeNode> queue = new ArrayDeque<>();
    queue.offer(root);

    while (!queue.isEmpty()) {
        int levelSize = queue.size();
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < levelSize; i++) {
            TreeNode node = queue.poll();
            level.add(node.val);
            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
        result.add(level);
    }
    return result;
}
```

Why `levelSize` Is Critical:

When the loop begins processing a level, the queue contains exactly the nodes at that level. Capturing `queue.size()` before the inner for-loop ensures you process

only those nodes, not the children you add during this iteration. Without `levelSize`, you would mix nodes from different levels.

Trace Through the Example Tree:

```
Initial queue: [4]

Level 0: levelSize = 1
  Poll 4, add children 2 and 7
  level = [4]
  Queue: [2, 7]

Level 1: levelSize = 2
  Poll 2, add children 1 and 3
  Poll 7, add children 6 and 9
  level = [2, 7]
  Queue: [1, 3, 6, 9]

Level 2: levelSize = 4
  Poll 1 (no children), Poll 3 (no children)
  Poll 6 (no children), Poll 9 (no children)
  level = [1, 3, 6, 9]
  Queue: []

Result: [[4], [2, 7], [1, 3, 6, 9]]
```

When to Use Level-Order:

Signal in Problem	Why Level-Order	Example
"Level by level" or "each level"	Directly processes levels	Level order traversal (LeetCode 102)
"Right side view"	Last node at each level	Binary tree right side view (LeetCode 199)
"Zigzag" or "spiral order"	Level-order with alternating direction	Zigzag level order (LeetCode 103)
"Minimum depth"	BFS finds the first leaf, which is the shallowest	Minimum depth of binary tree (LeetCode 111)
"Average of levels"	Need all values at each level	Average of levels (LeetCode 637)

Time O(n), space O(w) where w is the maximum width of the tree. For a complete binary tree, the last level has up to n/2 nodes, so the queue holds up to O(n)

nodes.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 142: Recursive Tree Patterns

Three Patterns That Solve Most Tree Problems

Nearly every recursive tree problem falls into one of three patterns based on how information flows through the tree. Recognizing which pattern applies is the key skill.

Pattern 1: Pass Information Down (Root to Leaf)

The parent passes context to its children as a parameter. Used when each node's answer depends on its ancestors.

```
public boolean hasPathSum(TreeNode node, int remainingSum) {
    if (node == null) return false;
    remainingSum -= node.val;
    if (node.left == null && node.right == null) {
        return remainingSum == 0;
    }
    return hasPathSum(node.left, remainingSum)
        || hasPathSum(node.right, remainingSum);
}
```

The running sum is passed down. Each node subtracts its value. At a leaf, check if the sum reached zero. Problems that use this pattern: root-to-leaf path sum, validate BST (pass valid range down), depth tracking.

Pattern 2: Return Information Up (Leaf to Root)

Each recursive call returns a value that the parent uses to compute its own answer. Used for bottom-up aggregation.

```
public int maxDepth(TreeNode node) {
    if (node == null) return 0;
    int leftDepth = maxDepth(node.left);
    int rightDepth = maxDepth(node.right);
    return 1 + Math.max(leftDepth, rightDepth);
}
```

Children report their depth upward. The parent takes the larger one and adds 1. Problems that use this pattern: maximum depth, tree height, subtree size, check if balanced.

Pattern 3: Combine Left and Right Subtree Results

The answer at each node requires information from both the left and right subtrees, combined at the current node. This often involves tracking a global result alongside the return value.

```
int diameter = 0;

public int diameterOfBinaryTree(TreeNode root) {
    height(root);
    return diameter;
}

private int height(TreeNode node) {
    if (node == null) return 0;
    int leftHeight = height(node.left);
    int rightHeight = height(node.right);
    diameter = Math.max(diameter, leftHeight + rightHeight);
    return 1 + Math.max(leftHeight, rightHeight);
}
```

Each node computes the longest path through itself (left height + right height) and updates the global maximum. But the function returns height, not diameter, because that is what the parent needs. Problems that use this pattern: diameter, longest path, maximum path sum.

Decision Table:

What the Problem Needs	Pattern	Information Flow	Example
Answer depends on ancestors	Pass down	Root to leaf	Path sum, validate BST, depth
Answer depends on descendants	Return up	Leaf to root	Max depth, height, subtree size
Answer combines both subtrees at a node	Combine	Both directions	Diameter, longest path, max path sum

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 143: Worked Example: Maximum Depth of Binary Tree

Problem: Maximum Depth of Binary Tree (LeetCode 104)

Given the root of a binary tree, return its maximum depth. Maximum depth is the number of nodes along the longest path from the root down to the farthest leaf.

Pattern Recognition: "Maximum depth" is a bottom-up aggregation. Each node's depth depends on its children's depths. This is Pattern 2: return information up.

Step-by-Step Approach:

Base case: if the node is null, the depth is 0 (no node, no depth contribution).

Recursive case: the depth at any node is 1 (for the node itself) plus the maximum depth of its left and right subtrees.

Full Java Solution:

```
public int maxDepth(TreeNode root) {
    if (root == null) return 0;
    int leftDepth = maxDepth(root.left);
    int rightDepth = maxDepth(root.right);
```

```
    return 1 + Math.max(leftDepth, rightDepth);
}
```

Three lines of logic. This is one of the cleanest examples of "trust the recursion": assume `maxDepth(root.left)` correctly returns the left subtree's depth, assume `maxDepth(root.right)` correctly returns the right subtree's depth, then combine with `1 + max`.

Trace Through the Example Tree:

```
        4
       / \
      2   7
     / \ / \
    1  3 6  9

maxDepth(4):
  maxDepth(2):
    maxDepth(1):
      maxDepth(null) = 0     (left of 1)
      maxDepth(null) = 0     (right of 1)
      return 1 + max(0, 0) = 1
    maxDepth(3):
      maxDepth(null) = 0
      maxDepth(null) = 0
      return 1 + max(0, 0) = 1
    return 1 + max(1, 1) = 2
  maxDepth(7):
    maxDepth(6):  returns 1   (same logic as node 1)
    maxDepth(9):  returns 1
    return 1 + max(1, 1) = 2
  return 1 + max(2, 2) = 3

Result: 3
```

Complexity:

- Time: O(n). Every node is visited exactly once.
- Space: O(h) where h is the height of the tree (recursion stack depth). For a balanced tree this is O(log n). For a skewed tree this is O(n).

Iterative Alternative (BFS):

Count the number of levels using level-order traversal. Each level processed increments a depth counter. The final counter value is the maximum depth.

```
public int maxDepth(TreeNode root) {
    if (root == null) return 0;
    Deque<TreeNode> queue = new ArrayDeque<>();
    queue.offer(root);
    int depth = 0;
    while (!queue.isEmpty()) {
        int levelSize = queue.size();
        for (int i = 0; i < levelSize; i++) {
            TreeNode node = queue.poll();
            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
        depth++;
    }
    return depth;
}
```

Same O(n) time, O(w) space where w is the max width. The recursive version is more concise; the iterative version avoids stack overflow on extremely deep trees. In an interview, the recursive version is preferred for clarity unless the interviewer asks for iterative.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 144: Binary Search Trees: Properties and Operations

The BST Property

A binary search tree is a binary tree where for every node: all values in the left subtree are strictly less than the node's value, and all values in the right subtree are strictly greater. This property holds recursively for every subtree.

```
Valid BST:          NOT a valid BST:
      5                   5
     / \                 / \
    3   8               3   8
   / \ / \             / \ / \
  1  4 7  9           1  6 7  9

Left tree: every node satisfies the BST property.
Right tree: node 6 is in the LEFT subtree of 5, but 6 > 5. Violates BST.
```

Why BSTs Matter:

Inorder traversal of a BST produces sorted order. This means searching, finding minimum/maximum, and finding the next larger/smaller element are all efficient.

BST Search: O(h)

At each node, compare the target with the current value. Go left if smaller, right if larger, return if equal.

```java
public TreeNode search(TreeNode root, int target) {
    if (root == null || root.val == target) return root;
    if (target < root.val) return search(root.left, target);
    return search(root.right, target);
}
```

BST Insertion: O(h)

Same navigation as search. When you reach a null position, that is where the new node belongs.

```java
public TreeNode insert(TreeNode root, int val) {
    if (root == null) return new TreeNode(val);
    if (val < root.val) {
        root.left = insert(root.left, val);
    } else {
        root.right = insert(root.right, val);
    }
```

```
        return root;
    }
```

BST Deletion: Three Cases

Deletion is more complex because removing a node with two children requires restructuring.

Case	Situation	Action
Leaf node	No children	Simply remove it (set parent's reference to null)
One child	Left or right child exists, not both	Replace node with its child
Two children	Both left and right children exist	Replace node's value with its inorder successor (smallest value in right subtree), then delete the successor from the right subtree

Deletion implementation is less commonly asked than search and insert at the fresher level, but you should be able to describe the three cases verbally.

BST Operation Complexity:

Operation	Average Case	Worst Case (Skewed)	Why
Search	O(log n)	O(n)	Balanced: halves search space. Skewed: linear chain.
Insert	O(log n)	O(n)	Same reasoning as search
Delete	O(log n)	O(n)	Find node + find successor, both O(h)
Inorder traversal	O(n)	O(n)	Always visits every node

The worst case O(n) happens when elements are inserted in sorted order, creating a tree that degenerates into a linked list. Balanced BSTs (AVL, red-black trees) guarantee O(log n) height but are rarely asked to implement in interviews. Know they exist and why they matter: they prevent the skewed worst case.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

SLIDE 145: Worked Example: Validate Binary Search Tree

Problem: Validate Binary Search Tree (LeetCode 98)

Given the root of a binary tree, determine if it is a valid BST.

The Wrong Approach That Many Candidates Use:

Check if each node is greater than its left child and less than its right child.

```
// WRONG: only checks immediate children
public boolean isValidBST(TreeNode root) {
    if (root == null) return true;
    if (root.left != null && root.left.val >= root.val) retur
n false;
    if (root.right != null && root.right.val <= root.val) ret
urn false;
    return isValidBST(root.left) && isValidBST(root.right);
}
```

Why It Fails:

```
      5
     / \
    1   6
       / \
      3   7
```

This tree passes the local check: 6 > 5 (right child of 5), 3 < 6 (left child of 6), 7 > 6 (right child of 6). But 3 is in the RIGHT subtree of 5, and 3 < 5. This violates the BST property. The local check misses it because it does not propagate the constraint from ancestor nodes.

The Correct Approach: Pass Valid Range Down (Pattern 1)

Each node must fall within a range determined by its ancestors. The root can be any value. The left child of 5 must be in range (-infinity, 5). The right child of 5

must be in range (5, +infinity). The left child of 6 (which is in the right subtree of 5) must be in range (5, 6).

```
public boolean isValidBST(TreeNode root) {
    return validate(root, Long.MIN_VALUE, Long.MAX_VALUE);
}

private boolean validate(TreeNode node, long min, long max) {
    if (node == null) return true;
    if (node.val <= min || node.val >= max) return false;
    return validate(node.left, min, node.val)
        && validate(node.right, node.val, max);
}
```

Using `long` for min and max handles the edge case where node values are Integer.MIN_VALUE or Integer.MAX_VALUE.

Trace Through the Invalid Tree:

```
validate(5, -INF, +INF): 5 in range → continue
  validate(1, -INF, 5): 1 in range → continue
    validate(null): return true
    validate(null): return true
    return true
  validate(6, 5, +INF): 6 in range → continue
    validate(3, 5, 6): 3 <= 5 → return FALSE

Result: false (node 3 violates the range constraint inherited from node 5)
```

The range [5, 6] for node 3's position means it must be greater than 5 AND less than 6. The value 3 fails the lower bound.

Alternative Approach: Inorder Traversal

Since inorder traversal of a valid BST produces sorted order, traverse the tree inorder and check that each value is strictly greater than the previous.

```
private long prev = Long.MIN_VALUE;
```

```java
public boolean isValidBST(TreeNode root) {
    if (root == null) return true;
    if (!isValidBST(root.left)) return false;
    if (root.val <= prev) return false;
    prev = root.val;
    return isValidBST(root.right);
}
```

Both approaches are O(n) time, O(h) space. The range approach is the stronger interview answer because it explicitly demonstrates the "pass information down" recursive pattern.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 146: Wrong vs Right: Tree Mistakes

Mistake 1: Checking Only Immediate Children for BST Validity

Wrong: Comparing a node only against its direct left and right children.

Right: Passing a valid range (min, max) down the tree. Every node must satisfy constraints from all its ancestors, not just its parent. Slide 145 demonstrates this with a full trace.

Mistake 2: Not Handling Null Properly

Wrong: Forgetting the null check, causing NullPointerException.

```java
// Crashes when root is null
public int maxDepth(TreeNode root) {
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.ri
ght));
}
```

Right: Null is the base case for every tree recursion. Always handle it first.

```
public int maxDepth(TreeNode root) {
    if (root == null) return 0;
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.ri
ght));
}
```

Every recursive tree function should start with `if (node == null)`. This is the equivalent of the base case rule from Section 12: "always define the base case first."

Mistake 3: Confusing Depth and Height

Wrong: Using "depth" and "height" interchangeably.

Right: Depth is measured from the root downward (root has depth 0). Height is measured from a node upward to its deepest leaf (a leaf has height 0). The height of the tree equals the maximum depth of any node. In LeetCode problems, "maximum depth" and "height of the tree" are the same numerical value, but the direction of measurement is different. When an interviewer says "depth of node X", they mean root-to-X distance. When they say "height", they mean X-to-deepest-leaf distance.

Mistake 4: Using Iteration When Recursion Is Cleaner

Wrong: Writing 20 lines of iterative code with an explicit stack for a problem that recursion solves in 5 lines.

Right: Default to recursion for tree problems. Switch to iterative only when stack overflow is a concern (extremely deep trees) or when the interviewer specifically asks. Most tree problems are cleaner recursively because the tree itself is recursive.

Mistake 5: Using Global Variables Unnecessarily

Wrong: Declaring class-level variables for tracking state that could be passed as parameters or returned.

Right: Pass context down as parameters (Pattern 1) or return results up (Pattern 2). Global variables make code harder to reason about and are not thread-safe. The diameter example (Slide 142) uses a global because it tracks a running maximum

alongside a return value, which is a legitimate use. But if information can flow through parameters or return values, prefer those.

Mistake 6: Converting Tree to Array and Solving on Array

Wrong: Collecting all values into a list, sorting, then answering the question.

Right: Solve the problem using the tree structure directly. Collecting all values loses the structural information (parent-child relationships, left/right placement) that the tree provides. An interviewer who gives you a tree problem expects a tree solution.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 147: How to Present Tree Solutions in an Interview

Draw the Tree

Before writing any code, draw a small example tree. Five to seven nodes is sufficient. This takes 15-20 seconds and provides a visual reference for both you and the interviewer. More mistakes come from incorrect mental models of tree structure than from incorrect code logic. Drawing prevents this.

Introducing a Recursive Tree Solution:

"I will solve this recursively. The base case is a null node, which returns [appropriate base value]. For each non-null node, I recursively compute the result for the left and right subtrees, then combine them by [specific operation]."

For maximum depth: "The base case is null, returning 0. For each node, I take the maximum depth of its left and right subtrees and add 1. This propagates the depth from leaves upward to the root."

For validate BST: "I will pass a valid range down to each node. The root can be anything. When I go left, I tighten the upper bound to the current node's value. When I go right, I tighten the lower bound. If any node falls outside its range, the tree is not a valid BST."

When to Mention the Iterative Alternative:

After presenting the recursive solution, briefly mention: "This can also be done iteratively with an explicit stack (for DFS) or queue (for BFS). The trade-off is the same O(n) time, but iterative avoids potential stack overflow on trees with depth exceeding Java's default stack size, which is roughly 5,000-10,000 frames."

Only implement the iterative version if the interviewer asks. Mentioning it shows awareness without derailing into unnecessary complexity.

Stating Complexity for Tree Problems:

"Time is O(n) because I visit every node exactly once. Space is O(h) for the recursion stack, where h is the height. For a balanced tree that is O(log n), for a skewed tree it is O(n) in the worst case."

Always clarify the space in terms of height, not just O(n). Saying "O(h) where h is the height" shows you understand that tree shape affects performance, not just tree size.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 148: Interview Practice: Trees

Practice Prompt 1

"Find the maximum depth of a binary tree." (LeetCode 104)

Expected Walkthrough:

Step 1: "I will solve this recursively. At each node, the depth is 1 plus the maximum depth of the left and right subtrees."

Step 2: Draw a small tree (5-7 nodes).

Step 3: Implement:

```
public int maxDepth(TreeNode root) {
    if (root == null) return 0;
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.ri
```

```
ght));
}
```

Step 4: Trace through the example tree, showing the recursion unwinding from leaves to root.

Step 5: "Time O(n), space O(h). For a balanced tree the space is O(log n), worst case O(n) for a skewed tree."

Follow-Up: "Now do it iteratively with BFS."

"I will use level-order traversal with a queue. Each time I finish processing a level, I increment a depth counter. The final counter value is the maximum depth."

Implement the BFS version from Slide 143. The interviewer is checking whether you can switch between recursive and iterative approaches and understand the trade-offs.

Practice Prompt 2

"Given a binary tree, check if it is symmetric (mirror image of itself)." (LeetCode 101)

Approach: Define a helper that takes two nodes. Base cases: both null (symmetric), one null (not symmetric). Recursive case: values must be equal, left's left must mirror right's right, and left's right must mirror right's left.

```
public boolean isSymmetric(TreeNode root) {
    return isMirror(root, root);
}

private boolean isMirror(TreeNode t1, TreeNode t2) {
    if (t1 == null && t2 == null) return true;
    if (t1 == null || t2 == null) return false;
    return t1.val == t2.val
        && isMirror(t1.left, t2.right)
        && isMirror(t1.right, t2.left);
}
```

Common Fresher Mistakes on Tree Problems:

- Not handling the null/empty tree case (returning incorrect value or crashing on root == null)
- Confusing depth (root to node, measured downward) with height (node to deepest leaf, measured upward)
- Overcomplicating with iterative code when a 3-5 line recursive solution exists
- Not drawing the tree before coding, leading to incorrect mental models about node relationships
- For BST problems: checking only the immediate parent-child relationship instead of the full range constraint from all ancestors

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 149: Section Cheat Sheet

Traversal Summary

Traversal	Order	Implementation	Primary Use
Inorder	Left, Root, Right	Recursion	Sorted order from BST
Preorder	Root, Left, Right	Recursion	Serialize/copy tree structure
Postorder	Left, Right, Root	Recursion	Bottom-up: height, size, delete
Level-order	Level by level, L to R	Queue (BFS)	Level-based problems, min depth

Recursive Pattern Decision Table

Information Flow	Pattern	When to Use	Example Problems
Root to leaf (top-down)	Pass information down as parameters	Answer depends on ancestors	Path sum, validate BST, depth at each node
Leaf to root (bottom-up)	Return information up as return values	Answer depends on descendants	Max depth, height, subtree size, is balanced

Information Flow	Pattern	When to Use	Example Problems
Both directions at each node	Combine left and right results	Answer uses both subtrees at a node	Diameter, longest path, max path sum

BST Operations and Complexity

Operation	Average	Worst (Skewed)	Technique
Search	O(log n)	O(n)	Compare and go left or right
Insert	O(log n)	O(n)	Search for null position, place node
Delete	O(log n)	O(n)	Three cases: leaf, one child, two children
Inorder traversal	O(n)	O(n)	Always visits all nodes

Pattern Recognition Signals

Signal in Problem	First Pattern to Try	Template
"Maximum/minimum depth"	Return info up (Pattern 2)	1 + max(left, right)
"Path sum", "root to leaf"	Pass info down (Pattern 1)	Subtract value, check at leaf
"Validate BST"	Pass range down (Pattern 1)	min/max bounds tighten at each level
"Diameter", "longest path"	Combine left + right (Pattern 3)	Track global max, return height
"Level order", "right side view"	BFS with queue	levelSize loop, process per level
"Sorted order from BST"	Inorder traversal	Left, process, right
"Symmetric / mirror"	Two-node recursion	Compare corresponding positions
"Serialize / deserialize"	Preorder traversal	Root first, then structure

Key Complexity Facts

Problem Type	Time	Space
Any single traversal	O(n)	O(h) recursion stack or O(w) queue
BST search/insert/delete	O(h), avg O(log n)	O(h) recursive, O(1) iterative

Problem Type	Time	Space
Problems visiting every node	O(n)	O(h) or O(n) depending on approach

Where h = height of tree. Balanced: h = O(log n). Skewed: h = O(n). Always state this distinction.

How This Connects to Other Sections

- Section 10 (Stacks and Queues): the queue-based BFS pattern from Slide 94 is applied directly as level-order traversal here. The same template (offer, levelSize loop, poll and process) carries over unchanged.
- Section 12 (Recursion and Backtracking): "trust the recursion" from Slide 111 is the mental model for all tree recursion. The base case (null node) and recursive case (left/right subtrees) follow the same anatomy.
- Section 16 (Graphs): tree traversals are special cases of graph traversals. BFS on a tree is BFS on a graph without cycles (no visited set needed). DFS on a tree is DFS on a graph without cycles. Section 16 adds the visited set and handles multiple components.
- Section 17 (Dynamic Programming): some tree DP problems (house robber on trees, max path sum) combine the recursive tree patterns here with memoization from Section 17. The "return info up" pattern is the foundation for tree DP.

Common Interview Mistakes

Mistake	Why It's Wrong	Better Approach
Local BST check (only immediate children)	Misses constraints from ancestors	Pass min/max range down the tree
No null base case	NullPointerException on empty trees or leaves	First line: `if (node == null) return [base value]`
Confusing depth and height	Wrong answers, imprecise communication	Depth = root to node (down), height = node to leaf (up)
Iterative when recursive is 3 lines	Overcomplicates, wastes interview time	Default to recursion, mention iterative as alternative
Collecting all values into a list	Loses tree structure, defeats the purpose	Solve using the tree directly

Mistake	Why It's Wrong	Better Approach
Not drawing the tree	Mental model errors in recursion	Draw 5-7 nodes before coding, trace through it

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina